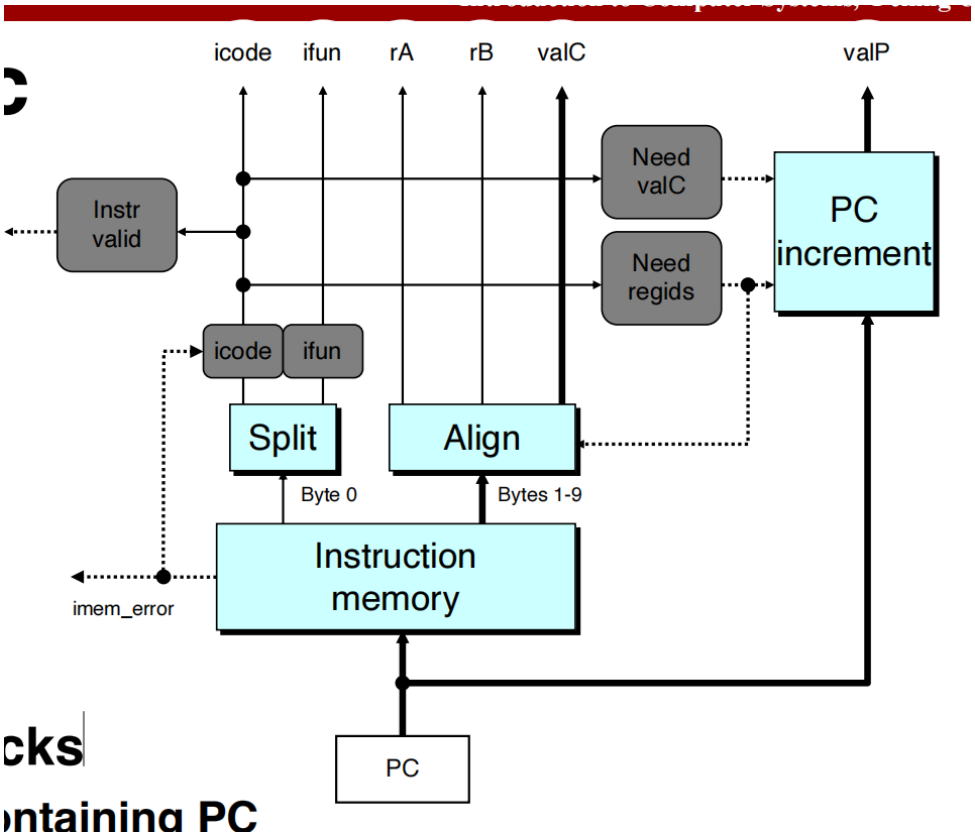
01 解读 Fetch 部件所有灰色框中信号的生成逻辑

基于课件第 30 页及前后相关内容



课件第 30 页 · Fetch

```
word icode = [
    imem_error : INOP;
    1 : imem_icode;
];

word ifun = [
    imem_error : FNONE;
    1 : imem_ifun;
];

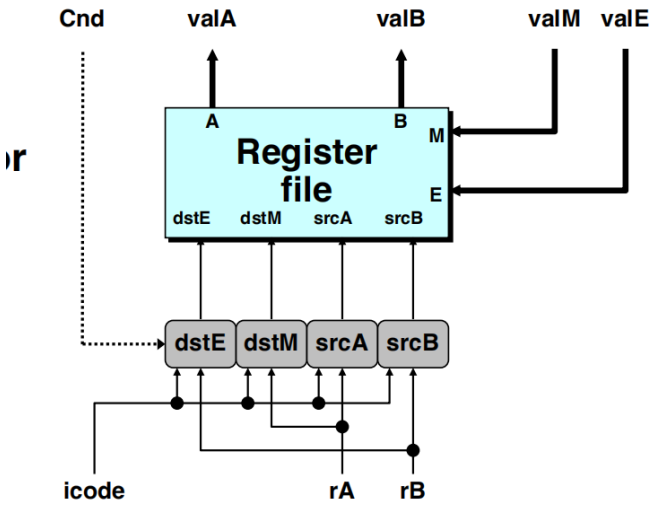
bool instr_valid = icode in {
    IHALT, INOP, IRRMOVQ, IIRMOVQ,
    IRMMOVQ, IMRMOVQ, IOPQ, IJXX,
    ICALL, IRET, IPUSHQ, IPOPQ
};

bool need_regids = icode in {
    IRRMOVQ, IIRMOVQ, IRMMOVQ, IMRMOVQ,
    IOPQ, IPUSHQ, IPOPQ
};

bool need_valC = icode in {
    IIRMOVQ, IRMMOVQ, IMRMOVQ, IJXX, ICALL
};
```

02 说明 Decode 部件所有信号的基本用途

基于课件第 34 页



课件第 34 页 · Decode

icode：指令类别，用来决定读写寄存器的规则。

rA：指令中编码的第一个寄存器编号。

rB：指令中编码的第二个寄存器编号。

srcA：A 读端口要读取的寄存器编号。

srcB：B 读端口要读取的寄存器编号。

valA：从 srcA 指定的寄存器读出的值。

valB：从 srcB 指定的寄存器读出的值。

dstE：指定把 valE 写入哪个寄存器。

dstM：指定把 valM 写入哪个寄存器。

valE：ALU 产生的结果，送往 E 写端口。

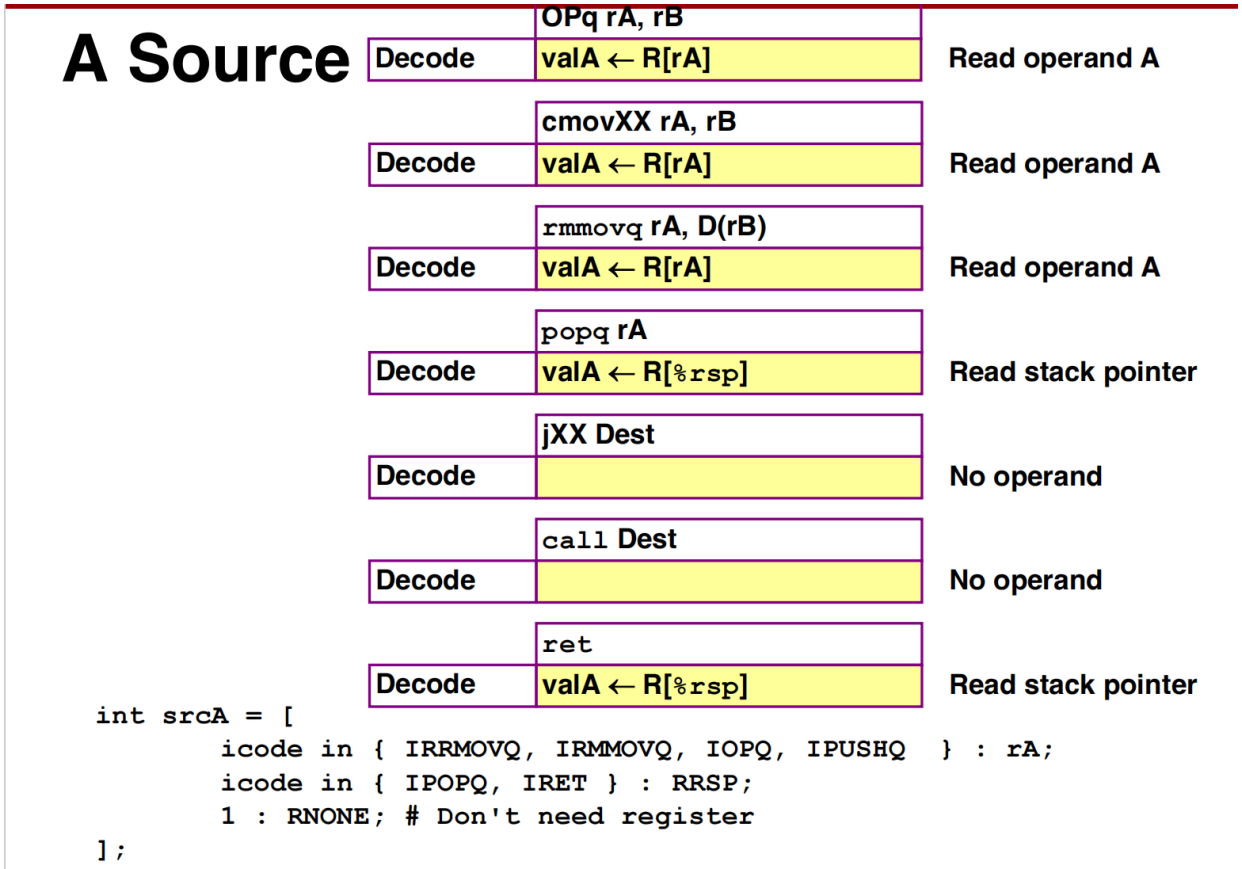
valM：内存读出的值，送往 M 写端口。

Cnd：表示条件是否成立。条件传送成立时 dstE = rB；否则 dstE = RNONE，取消写入。

记住：src / dst 指定“哪个寄存器”；val 表示“寄存器或存储器中的数据”。

03 解读 srcA 的生成逻辑，写出并分析 srcB 的生成逻辑

先解读课件第 35 页，再补充 srcB

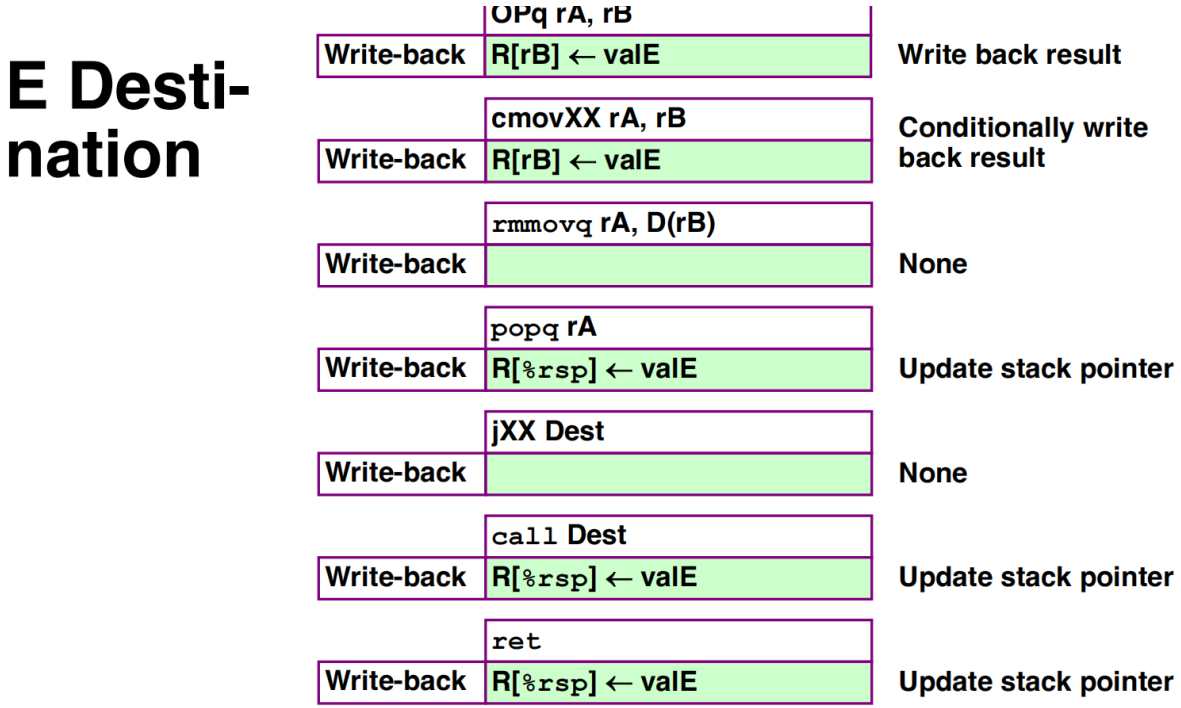


课件第 35 页 · A Source

```
word srcB = [
    icode in {
        IRMMOVQ, IMRMOVQ, IOPQ
    } : rB;
    icode in {
        IPUSHQ, IPOPQ, ICALL, IRET
    } : RRSP;
    1 : RNONE;
];
```

04 解读 dstE 的生成逻辑，写出并分析 dstM 的生成逻辑

先解读课件第 36 页，再补充 dstM




```
int dstE = [
        icode in { IRRMOVQ } && Cnd : rB;
        icode in { IIRMOVQ, IOPQ} : rB;
        icode in { IPUSHQ, IPOPQ, ICALL, IRET } : RRSP;
        1 : RNONE;  # Don't write any register
];
```

课件第 36 页 · E Destination

```
word dstM = [
    icode in { IMRMOVQ, IPOPQ } : rA;
    1 : RNONE;
];
```

05 解读 ALU A 的生成逻辑，写出并分析 ALU B 的生成逻辑

先解读课件第 38 页，再补充 ALU B

ALU A Input

	OPq rA, rB	
Execute	valE ← valB OP valA	Perform ALU operation
	cmovXX rA, rB	
Execute	valE ← 0 + valA	Pass valA through ALU
	rmmovq rA, D(rB)	
Execute	valE ← valB + valC	Compute effective address
	popq rA	
Execute	valE ← valB + 8	Increment stack pointer
	jXX Dest	
Execute		No operation
	call Dest	
Execute	valE ← valB + –8	Decrement stack pointer
	ret	
Execute	valE ← valB + 8	Increment stack pointer

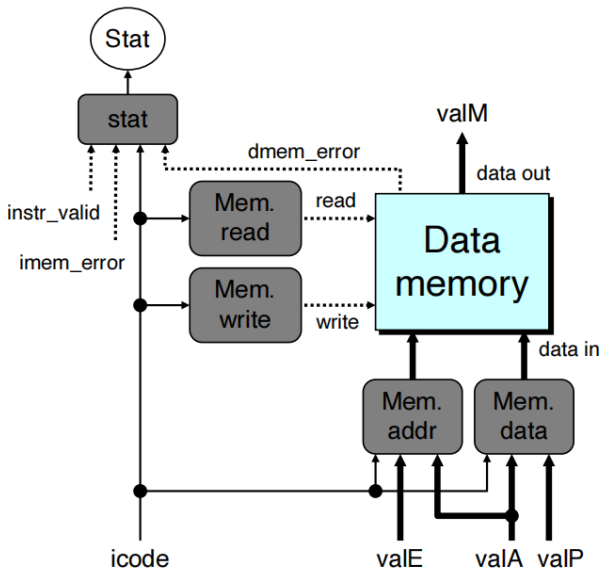
```
int aluA = [
        icode in { IRRMOVQ, IOPQ } : valA;
        icode in { IIRMOVQ, IRMMOVQ, IMRMOVQ } : valC;
        icode in { ICALL, IPUSHQ } : -8;
        icode in { IRET, IPOPQ } : 8;
        # Other instructions don't need ALU
];
```

课件第 38 页 · ALU A Input

```
word aluB = [
    icode in {
        IRRMOVQ, IIRMOVQ
    } : 0;
    icode in {
        IRMMOVQ, IMRMOVQ, IOPQ,
        ICALL, IPUSHQ, IRET, IPOPQ
    } : valB;
    1 : 0;
];
```

06 说明 Memory 部件所有信号的基本用途

基于课件第 40 页



课件第 40 页 · Memory

icode：指令类别，供逻辑生成访存控制信号。

mem_read：读使能，为 1 时读取数据存储器。

mem_write：写使能，为 1 时写入数据存储器。

mem_addr：本次读或写的地址，选择 valE 或 valA。

mem_data：本次写入的数据，选择 valA 或 valP。

valE：ALU 结果，可作为有效地址或压栈后的新栈顶。

valA：待存数据；对 popq、ret 则是旧栈指针。

valP：顺序下一条地址，call 把它作为返回地址压栈。

valM：从数据存储器读出的数据。

imem_error：指令存储器报告的取指访问错误。

dmem_error：数据存储器报告的访问错误。

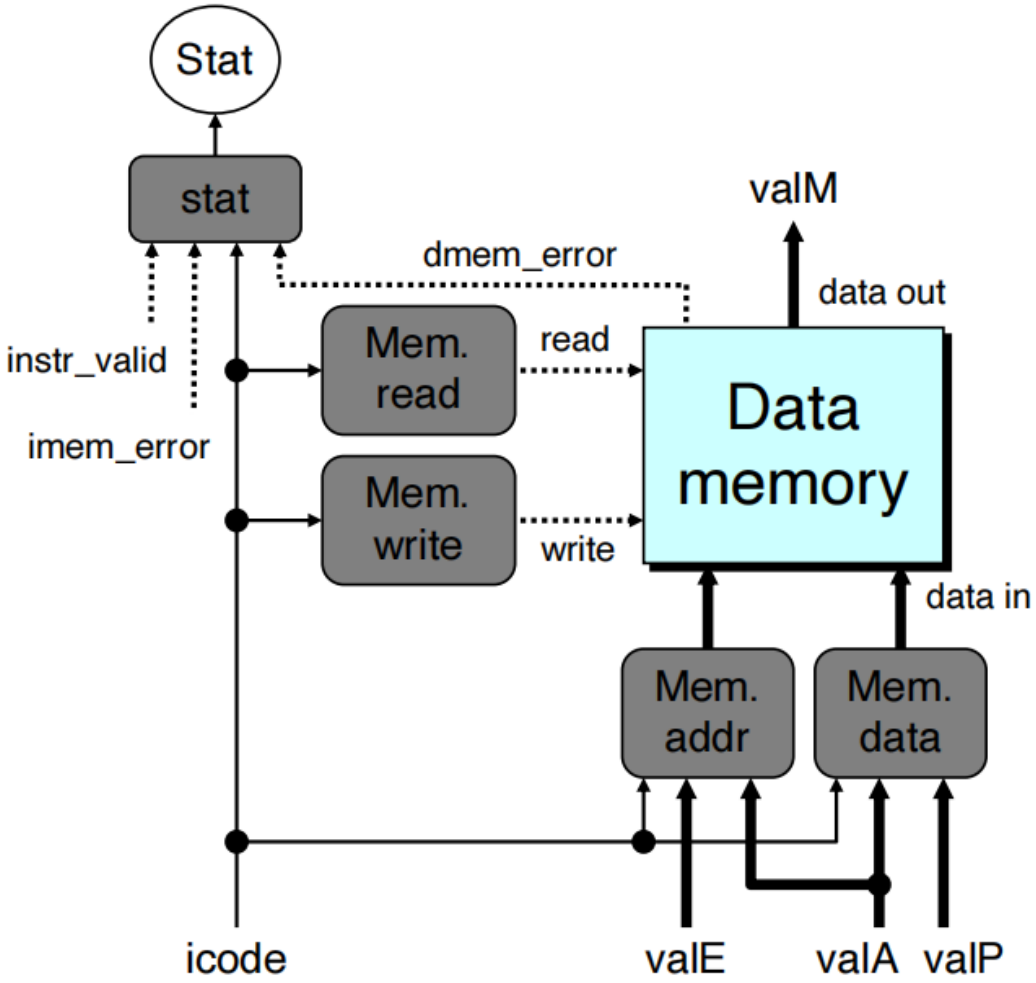
instr_valid：指令类别有效为 1，无效为 0。

stat：综合错误信号和 icode，按地址错误 → 非法指令 → 停机 → 正常的优先级生成状态。

基本状态：ADR 地址错误，INS 非法指令，HLT 停机，AOK 正常。

07 写出并分析 Memory Data 和 Memory Write 的生成逻辑

结合课件第 42、43 页



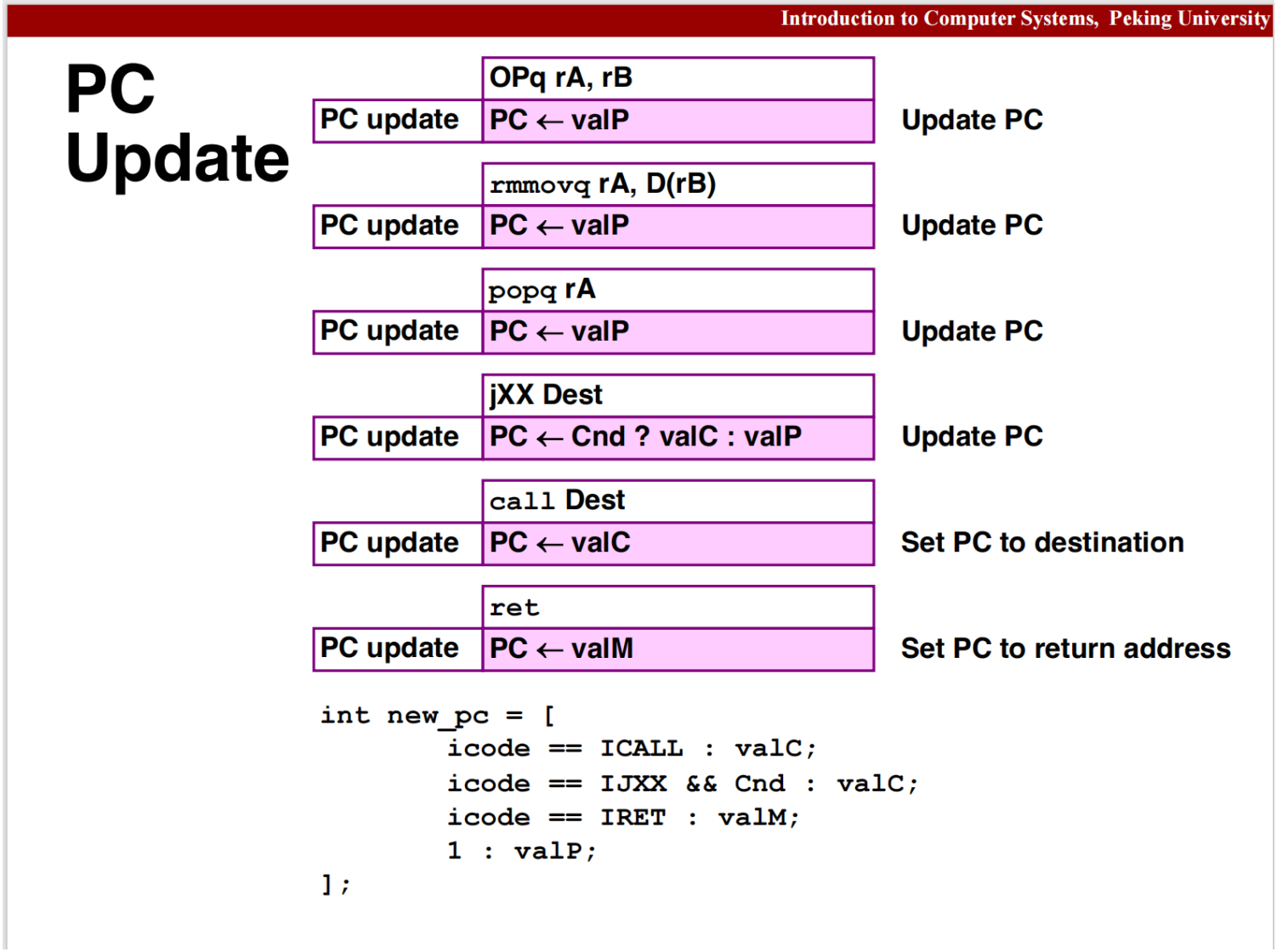
相关配图：已提供的第 40 页 Memory 图

```
word mem_data = [
    icode in {
        IRMMOVQ, IPUSHQ
    } : valA;
    icode == ICALL : valP;
    1 : 0;
];

bool mem_write = icode in {
    IRMMOVQ, IPUSHQ, ICALL
};
```

08 解读 PC 更新的生成逻辑

基于课件第 46 页



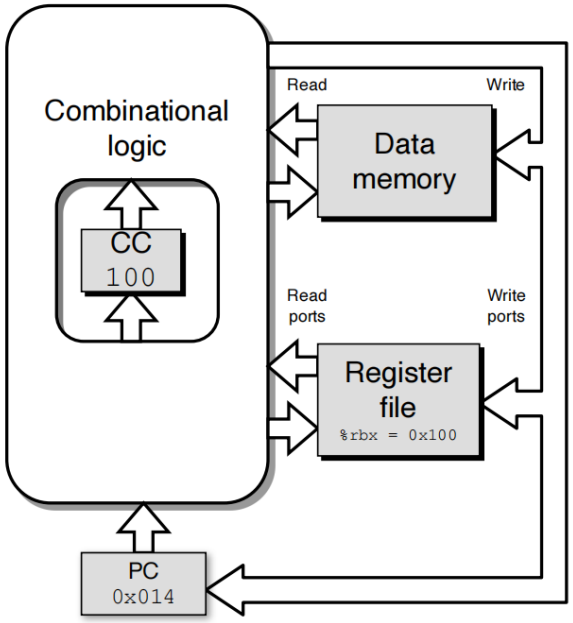
课件第 46 页 · PC Update

09 讲解 SEQ 执行一条指令的完整过程，以及存储部件的变化时间点和内容变化情况

基于课件第 46~50 页；存储相关部件指时序逻辑电路

执行过程：取指 → 译码 → 执行 → 访存 → 写回 → PC 更新，全部在一个较长时钟周期内完成。

更新时间：取指、译码、执行和访存读取在周期内完成；下一上升沿按需写入内存、写回寄存器、更新 CC 和 PC。



已提供配图 · SEQ 组合逻辑与存储部件

例：popq %rax

取指：取得指令和寄存器编号，rA 指定 %rax；计算 valP = PC + 2。

译码：读取旧栈指针，valA = valB = R[%rsp]。

执行：计算新栈指针，valE = valB + 8。

访存（周期内）：从旧栈顶读取数据，valM = M8[valA]。

写回（下一上升沿）：同时进行 R[%rsp] ← valE、R[%rax] ← valM。

PC 更新（同一上升沿）：PC ← valP。

保持不变：CC 和数据存储器中的内容。